

Playwright API Testing

& Web Services — Complete Study Material

Topics Covered

Module 1	Overview of Playwright
Module 2	Basic Playwright API Testing
Module 3	Validations & Assertions
Module 4	Authentication & Authorization
Module 5	Test Data Handling
Module 6	CI/CD Integration
Module 7	Web Application Fundamentals
Module 8	HTTP, JSON & APIs
Module 9	Postman

Module 1: Overview of Playwright

1.1 What is Playwright?

Playwright is an open-source automation framework developed by Microsoft for end-to-end testing of web applications. It supports multiple browsers — Chromium, Firefox, and WebKit — using a single unified API. While widely known for UI automation, Playwright also provides powerful support for API testing through its `APIRequestContext`.

1.2 Key Features

- **Cross-browser support:** Chromium, Firefox, WebKit (Safari engine)
- **Multi-language:** JavaScript/TypeScript, Python, Java, C#
- **Auto-wait:** Automatically waits for elements to be ready
- **Network interception:** Intercept and modify HTTP requests/responses
- **Built-in test runner:** `@playwright/test` with fixtures, parallel execution
- **API testing:** First-class HTTP request support via `APIRequestContext`
- **Tracing & debugging:** Trace viewer, screenshots, videos
- **CI/CD friendly:** Headless mode, Docker support, GitHub Actions integration

1.3 Why Playwright for API Testing?

Playwright bridges UI and API testing in a single framework, allowing teams to:

- Test APIs independently without a browser
- Combine UI actions with API calls in the same test
- Reuse authentication state across requests
- Leverage the same assertion library (`expect`) for both UI and API tests
- Run in CI/CD pipelines with zero configuration

Feature	Playwright	Postman	RestAssured
Language	JS/TS/Python/Java/C#	GUI / JS scripts	Java
UI + API in 1	Yes	No	No
Built-in Assertions	Yes (<code>expect</code>)	Yes (<code>pm.expect</code>)	Yes (BDD)
CI/CD Integration	Excellent	Newman CLI	Maven/Gradle
Free/Open Source	Yes	Free tier only	Yes

Module 2: Basic Playwright API Testing

2.1 Playwright APIRequestContext

APIRequestContext is a Playwright class that enables sending HTTP requests. It provides methods like **get()**, **post()**, **put()**, **patch()**, and **delete()** — mirroring standard HTTP verbs.

- Available via **request** fixture in `@playwright/test`
- Supports cookies, headers, authentication, and base URL configuration
- Can be created globally or per-test
- Automatically handles JSON serialization/deserialization

2.2 Creating Request Context

There are two main ways to create a request context:

A. Using the built-in request fixture (recommended)

```
import { test, expect } from '@playwright/test';

test('API test using request fixture', async ({ request }) => {
  const response = await request.get('https://api.example.com/users');
  expect(response.status()).toBe(200);
});
```

B. Creating context manually via `playwright.request.newContext()`

```
import { test, expect, request } from '@playwright/test';

test('API test with custom context', async () => {
  const apiContext = await request.newContext({
    baseURL: 'https://api.example.com',
    extraHTTPHeaders: {
      'Authorization': 'Bearer my-token',
      'Content-Type': 'application/json'
    }
  });
  const response = await apiContext.get('/users');
  expect(response.status()).toBe(200);
  await apiContext.dispose(); // Always dispose when done
});
```

2.3 CRUD Operations Using Playwright

CREATE — POST Request

```
test('Create a new user - POST', async ({ request }) => {  
  const response = await request.post('https://reqres.in/api/users', {  
    data: {  
      name: 'John Doe',  
      job: 'Software Engineer'  
    }  
  });  
  expect(response.status()).toBe(201);  
  const body = await response.json();  
  console.log('Created user ID:', body.id);  
});
```

READ — GET Request

```
test('Get user by ID - GET', async ({ request }) => {  
  const response = await request.get('https://reqres.in/api/users/2');  
  expect(response.status()).toBe(200);  
  const body = await response.json();  
  expect(body.data.id).toBe(2);  
});
```

UPDATE — PUT Request

```
test('Update user - PUT', async ({ request }) => {  
  const response = await request.put('https://reqres.in/api/users/2', {  
    data: { name: 'Jane Doe', job: 'QA Engineer' }  
  });  
  expect(response.status()).toBe(200);  
  const body = await response.json();  
  expect(body.name).toBe('Jane Doe');  
});
```

UPDATE — PATCH Request

```
test('Partial update - PATCH', async ({ request }) => {  
  const response = await request.patch('https://reqres.in/api/users/2', {  
    data: { job: 'Senior QA' }  
  });  
  expect(response.status()).toBe(200);  
});
```

DELETE — DELETE Request

```
test('Delete user - DELETE', async ({ request }) => {  
  const response = await request.delete('https://reqres.in/api/users/2');
```

```
expect(response.status()).toBe(204);
});
```

2.4 Passing Headers

```
test('Request with custom headers', async ({ request }) => {
  const response = await request.get('https://api.example.com/data', {
    headers: {
      'Authorization': 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...',
      'Content-Type': 'application/json',
      'Accept': 'application/json',
      'X-Custom-Header': 'my-value',
      'X-Request-ID': '12345'
    }
  });
  expect(response.ok()).toBeTruthy();
});
```

2.5 Passing Query Parameters

Query parameters can be appended directly to the URL or passed via the **params** option:

```
// Option 1: Inline in URL
const response1 = await request.get(
  'https://api.example.com/users?page=1&per_page=10&sort=asc'
);

// Option 2: Using params object (recommended - auto URL-encodes)
const response2 = await request.get('https://api.example.com/users', {
  params: {
    page: 1,
    per_page: 10,
    sort: 'asc',
    filter: 'active'
  }
});
```

2.6 Passing Path Parameters

Playwright does not have a dedicated path parameter feature — they are embedded directly into the URL string:

```
const userId = 42;
const postId = 7;
```

```
// Construct URL with template literals
const response = await request.get(`https://api.example.com/users/${userId}/posts/${postId}`);

// Or build dynamically
const baseUrl = 'https://api.example.com';
const endpoint = `/users/${userId}`;
const response2 = await request.get(baseUrl + endpoint);
```

2.7 Passing Request Body

```
// JSON body (most common)
const response = await request.post('https://api.example.com/products', {
  data: {
    name: 'Laptop',
    price: 999.99,
    category: 'Electronics'
  }
});

// Form data (application/x-www-form-urlencoded)
const response2 = await request.post('https://api.example.com/login', {
  form: {
    username: 'testuser',
    password: 'secret123'
  }
});

// Multipart form data (file upload)
const response3 = await request.post('https://api.example.com/upload', {
  multipart: {
    file: {
      name: 'document.pdf',
      mimeType: 'application/pdf',
      buffer: fs.readFileSync('document.pdf')
    },
    description: 'Uploaded file'
  }
});
```

2.8 Handling API Responses

The response object returned by Playwright's API methods provides rich access to all response data:

```
const response = await request.get('https://api.example.com/users/1');

// Status
const status = response.status();          // e.g. 200
const statusText = response.statusText();  // e.g. "OK"
const isOk = response.ok();               // true if 200-299

// Headers
const headers = response.headers();        // all headers as object
const contentType = response.headers()['content-type'];

// Body
const jsonBody = await response.json();    // parse as JSON
const textBody = await response.text();    // raw string
const bufferBody = await response.body();  // raw Buffer

// URL
const url = response.url();
```

Module 3: Validations & Assertions

3.1 Playwright expect Assertions

Playwright uses the **expect** function from `@playwright/test`, inspired by Jest. It supports both synchronous and async assertions and provides detailed error messages on failure.

3.2 Validating Status Codes

```
test('Status code assertions', async ({ request }) => {  
  const response = await request.get('https://reqres.in/api/users/2');  
  
  // Exact status code  
  expect(response.status()).toBe(200);  
  
  // Check success range (200-299)  
  expect(response.ok()).toBeTruthy();  
  
  // NOT found  
  const notFound = await request.get('https://reqres.in/api/unknown/999');  
  expect(notFound.status()).toBe(404);  
});
```

3.3 Validating Response Body Values

```
test('Response body assertions', async ({ request }) => {  
  const response = await request.get('https://reqres.in/api/users/2');  
  const body = await response.json();  
  
  // Exact value  
  expect(body.data.id).toBe(2);  
  expect(body.data.first_name).toBe('Janet');  
  
  // Type check  
  expect(typeof body.data.email).toBe('string');  
  
  // Array checks  
  const listResponse = await request.get('https://reqres.in/api/users?page=1');  
  const listBody = await listResponse.json();  
  expect(listBody.data).toHaveLength(6);  
});
```

```

    expect(listBody.data[0]).toHaveProperty('id');

    // Contains substring
    expect(body.data.email).toContain('@');

    // Greater / Less than
    expect(listBody.total).toBeGreaterThan(0);
  });

```

3.4 Validating Response Headers

```

test('Response header assertions', async ({ request }) => {
  const response = await request.get('https://reqres.in/api/users');

  // Check Content-Type
  expect(response.headers()['content-type']).toContain('application/json');

  // Header exists
  expect(response.headers()).toHaveProperty('content-type');

  // Exact header value
  expect(response.headers()['access-control-allow-origin']).toBe('*');
});

```

3.5 Validating Response Time

```

test('Response time validation', async ({ request }) => {
  const start = Date.now();
  const response = await request.get('https://reqres.in/api/users');
  const duration = Date.now() - start;

  expect(response.status()).toBe(200);
  // Response should be under 3000ms
  expect(duration).toBeLessThan(3000);
  console.log(`Response time: ${duration}ms`);
});

```

3.6 Partial Response Validation

Use **expect.objectContaining()** to validate only specific fields without requiring an exact full-match:

```

test('Partial response validation', async ({ request }) => {
  const response = await request.get('https://reqres.in/api/users/2');

```

```

const body = await response.json();

// Only validate specific fields (ignore others)
expect(body.data).toMatchObject({
  id: 2,
  first_name: 'Janet'
});

// Validate array contains at least one matching element
const listResponse = await request.get('https://reqres.in/api/users?page=1');
const listBody = await listResponse.json();

expect(listBody.data).toEqual(
  expect.arrayContaining([
    expect.objectContaining({ id: 1 })
  ])
);
});

```

Assertion	Purpose
expect(x).toBe(y)	Strict equality (===)
expect(x).toEqual(y)	Deep equality (objects/arrays)
expect(x).toMatchObject(y)	Partial object match
expect(x).toContain(y)	String/array contains value
expect(x).toHaveProperty(key)	Object has a specific key
expect(x).toHaveLength(n)	Array/string length
expect(x).toBeTruthy()	Truthy value check
expect(x).toBeGreaterThan(n)	Numeric comparison
expect(x).toBeLessThan(n)	Numeric comparison
expect(x).toBeNull()	Null check

Module 4: Authentication & Authorization

4.1 Basic Authentication

Basic Auth encodes username:password in Base64 and sends it in the Authorization header.

```
test('Basic Authentication', async ({ request }) => {  
  // Method 1: Encode manually  
  const credentials = Buffer.from('username:password').toString('base64');  
  const response = await request.get('https://api.example.com/protected', {  
    headers: {  
      'Authorization': `Basic ${credentials}`  
    }  
  });  
  
  // Method 2: Use Playwright's built-in httpCredentials  
  const context = await request.newContext({  
    httpCredentials: {  
      username: 'admin',  
      password: 'password123'  
    }  
  });  
  const response2 = await context.get('https://api.example.com/protected');  
  expect(response2.status()).toBe(200);  
});
```

4.2 Bearer Token Authentication

```
test('Bearer Token Authentication', async ({ request }) => {  
  // Step 1: Login to get the token  
  const loginResponse = await request.post('https://reqres.in/api/login', {  
    data: {  
      email: 'eve.holt@reqres.in',  
      password: 'cityslicka'  
    }  
  });  
  expect(loginResponse.status()).toBe(200);  
  const { token } = await loginResponse.json();  
  
  // Step 2: Use token in subsequent requests  
  const protectedResponse = await request.get('https://api.example.com/profile', {
```

```

    headers: {
      'Authorization': `Bearer ${token}`
    }
  });
  expect(protectedResponse.status()).toBe(200);
});

```

4.3 OAuth 2.0

OAuth 2.0 is an industry-standard authorization framework. The most common flow for API testing is the **Client Credentials** flow:

```

test('OAuth 2.0 Client Credentials Flow', async ({ request }) => {
  // Step 1: Get access token from Authorization Server
  const tokenResponse = await request.post('https://auth.example.com/oauth/token', {
    form: {
      grant_type: 'client_credentials',
      client_id: 'your-client-id',
      client_secret: 'your-client-secret',
      scope: 'read write'
    }
  });
  expect(tokenResponse.status()).toBe(200);
  const tokenData = await tokenResponse.json();
  const accessToken = tokenData.access_token;

  // Step 2: Use access token to call protected API
  const apiResponse = await request.get('https://api.example.com/resource', {
    headers: {
      'Authorization': `Bearer ${accessToken}`,
      'Content-Type': 'application/json'
    }
  });
  expect(apiResponse.status()).toBe(200);
});

```

4.4 API Key Authentication

```

test('API Key Authentication', async ({ request }) => {
  const API_KEY = process.env.API_KEY || 'your-api-key-here';

  // Method 1: API key in headers
  const response1 = await request.get('https://api.example.com/data', {

```

```
    headers: { 'X-API-Key': API_KEY }
  });

  // Method 2: API key as query parameter
  const response2 = await request.get('https://api.example.com/data', {
    params: { api_key: API_KEY }
  });

  // Method 3: API key in Authorization header
  const response3 = await request.get('https://api.example.com/data', {
    headers: { 'Authorization': `ApiKey ${API_KEY}` }
  });

  expect(response1.status()).toBe(200);
});
```

Module 5: Test Data Handling

5.1 Static vs Dynamic Test Data

Aspect	Static Data	Dynamic Data
Definition	Hardcoded fixed values	Generated at runtime
Example	userId: 1, name: 'John'	UUID, timestamp, random email
Pros	Simple, predictable	Avoids conflicts, realistic
Cons	May clash in parallel tests	Harder to debug
Best for	Read operations (GET)	Write operations (POST/PUT)

```
// Dynamic data generation
import { faker } from '@faker-js/faker';

const dynamicUser = {
  name: faker.person.fullName(),
  email: faker.internet.email(),
  phone: faker.phone.number(),
  address: faker.location.streetAddress()
};
```

5.2 Passing Data Between API Calls

```
test('Pass data between API calls', async ({ request }) => {
  // Step 1: Create a resource and capture the ID
  const createResponse = await request.post('https://reqres.in/api/users', {
    data: { name: 'Test User', job: 'QA' }
  });
  const created = await createResponse.json();
  const userId = created.id; // <-- capture the created ID

  // Step 2: Use the captured ID in the next call
  const getResponse = await request.get(`https://reqres.in/api/users/${userId}`);

  // Step 3: Update using captured ID
  const updateResponse = await request.put(`https://reqres.in/api/users/${userId}`, {
    data: { name: 'Updated User' }
  });

  expect(updateResponse.status()).toBe(200);
});
```

```
});
```

5.3 Chaining APIs

```
test('API chaining: Login -> Get Profile -> Update', async ({ request }) => {  
  // 1. Login  
  const loginResp = await request.post('https://reqres.in/api/login', {  
    data: { email: 'eve.holt@reqres.in', password: 'cityslicka' }  
  });  
  const { token } = await loginResp.json();  
  
  // 2. Fetch user profile using token  
  const profileResp = await request.get('https://reqres.in/api/users/2', {  
    headers: { 'Authorization': `Bearer ${token}` }  
  });  
  const profile = await profileResp.json();  
  const email = profile.data.email;  
  
  // 3. Update using profile data  
  const updateResp = await request.patch('https://reqres.in/api/users/2', {  
    headers: { 'Authorization': `Bearer ${token}` },  
    data: { email: email, job: 'Senior Engineer' }  
  });  
  expect(updateResp.status()).toBe(200);  
});
```

5.4 Using Environment Variables

```
// playwright.config.ts  
import { defineConfig } from '@playwright/test';  
  
export default defineConfig({  
  use: {  
    baseURL: process.env.BASE_URL || 'https://reqres.in',  
    extraHTTPHeaders: {  
      'Authorization': `Bearer ${process.env.API_TOKEN}`  
    }  
  }  
});  
  
// In test file  
test('Use env variables', async ({ request }) => {
```

```

const baseUrl = process.env.BASE_URL;
const response = await request.get(`${baseUrl}/api/users`);
expect(response.ok()).toBeTruthy();
});

// Running with env vars:
// BASE_URL=https://staging.api.com API_TOKEN=xyz npx playwright test

```

5.5 External Test Data (JSON Files)

```

// testdata/users.json
[
  { "name": "Alice", "job": "Developer" },
  { "name": "Bob", "job": "Designer" }
]

// test file
import testData from './testdata/users.json';
import fs from 'fs';

test('Create users from JSON file', async ({ request }) => {
  const users = JSON.parse(fs.readFileSync('./testdata/users.json', 'utf-8'));

  for (const user of users) {
    const response = await request.post('https://reqres.in/api/users', {
      data: user
    });
    expect(response.status()).toBe(201);
    console.log(`Created: ${user.name}`);
  }
});

```

5.6 Integration with CI/CD

Playwright integrates seamlessly with popular CI/CD platforms. Below is a GitHub Actions example:

```

# .github/workflows/api-tests.yml
name: Playwright API Tests

on:
  push:
    branches: [main, develop]
  pull_request:

```

```
branches: [main]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Setup Node.js
```

```
        uses: actions/setup-node@v3
```

```
        with:
```

```
          node-version: '18'
```

```
      - name: Install dependencies
```

```
        run: npm ci
```

```
      - name: Install Playwright
```

```
        run: npx playwright install --with-deps
```

```
      - name: Run API tests
```

```
        run: npx playwright test
```

```
        env:
```

```
          BASE_URL: ${ secrets.BASE_URL }
```

```
          API_TOKEN: ${ secrets.API_TOKEN }
```

```
      - name: Upload test report
```

```
        uses: actions/upload-artifact@v3
```

```
        if: always()
```

```
        with:
```

```
          name: playwright-report
```

```
          path: playwright-report/
```

Module 7: Introduction to Web Applications

7.1 Types of Applications

Type	Description	Examples
Web Application	Runs in browser, accessible via URL	Gmail, YouTube, Facebook
Desktop Application	Installed locally on OS	MS Word, VLC, VS Code
Mobile Application	Runs on smartphones/tablets	WhatsApp, Instagram
Web Services	Backend APIs consumed by other apps	REST API, SOAP, GraphQL
Cloud Applications	Hosted on cloud infrastructure	AWS apps, Salesforce

7.2 Architecture of a Web Application

A typical modern web application follows a 3-tier or N-tier architecture:

- **Presentation Layer (Frontend):** UI rendered in browser — HTML, CSS, JavaScript, React/Angular/Vue
- **Application Layer (Backend/Business Logic):** Server-side logic — Node.js, Java Spring, Python Django, .NET
- **Data Layer (Database):** Persistent storage — MySQL, PostgreSQL, MongoDB, Redis

Communication between layers happens over HTTP/HTTPS using REST APIs or GraphQL. The flow is: **Client → Web Server → App Server → Database → App Server → Web Server → Client.**

7.3 What is a URL?

A **URL (Uniform Resource Locator)** is the address used to access a resource on the internet.

URL Syntax

```
https://api.example.com:8080/users/42?filter=active&page;=1#section2
```

scheme	authority	port	path	query params	fragment

Component	Example	Description
Scheme	https://	Protocol (http, https, ftp)
Authority	api.example.com	Domain name or IP address
Port	:8080	Network port (optional)
Path	/users/42	Resource location on server
Query	?filter=active&page=1	Key-value parameters
Fragment	#section2	In-page anchor (not sent to server)

Module 8: JSON, HTTP & APIs

8.1 What is JSON?

JSON (JavaScript Object Notation) is a lightweight, text-based data interchange format. It is human-readable, language-independent, and widely used in REST APIs.

Characteristics of JSON

- Text-based format — stored and transmitted as plain text
- Language-independent — supported by virtually all programming languages
- Self-describing — key names describe the data
- Hierarchical — supports nested objects and arrays
- Data types: string, number, boolean, null, object, array

```
{
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "isActive": true,
    "score": 98.5,
    "roles": ["admin", "editor"],
    "address": {
      "city": "New York",
      "zip": "10001"
    },
    "phone": null
  }
}
```

8.2 HTTP Request & Response Structure

HTTP Request Components

Component	Description	Example
Request Line	Method + URL + HTTP version	POST /api/users HTTP/1.1
Host Header	Target server	Host: api.example.com
Headers	Metadata about the request	Content-Type: application/json
Body	Data sent to server	{ "name": "Alice" }

HTTP Methods

Method	Purpose	Body?	Idempotent?
GET	Retrieve a resource	No	Yes
POST	Create a new resource	Yes	No
PUT	Replace a resource	Yes	Yes
PATCH	Partial update	Yes	No
DELETE	Remove a resource	No	Yes
HEAD	Headers only, no body	No	Yes
OPTIONS	List allowed methods	No	Yes

HTTP Status Codes

Range	Category	Common Codes
1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved, 302 Found, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

8.3 Layers of Web Application / SOA Testing

SOA (Service-Oriented Architecture) structures applications as a collection of loosely coupled services. Testing occurs at multiple layers:

Layer	What is Tested	Tools
UI Layer	End-to-end user workflows	Playwright, Selenium, Cypress
API Layer	Business logic, contracts	Playwright, Postman, RestAssured
Integration Layer	Service-to-service communication	Contract testing, Pact
Database Layer	Data integrity, queries	SQL scripts, DbUnit
Security Layer	Auth, injection, encryption	OWASP ZAP, Burp Suite

8.4 Introduction to API

An **API (Application Programming Interface)** is a set of defined rules that allows software applications to communicate with each other. In web context, APIs expose backend functionality over HTTP so clients (web, mobile, other services) can interact with server resources.

Types of API Testing

- **Functional Testing:** Verify the API does what it's supposed to do (correct data, correct status)
- **Load/Performance Testing:** Check behavior under high traffic (JMeter, k6)
- **Security Testing:** Auth bypass, injection attacks, token expiration
- **Contract Testing:** Ensure the API matches its documented contract (Pact)
- **Regression Testing:** Ensure new changes don't break existing behavior
- **Integration Testing:** Test how the API integrates with databases and other services
- **Negative Testing:** Test with invalid inputs — wrong types, missing fields, out-of-range values

8.5 Web Services

A **Web Service** is a standardized way of communication between applications over a network using open standards (HTTP, XML, JSON). It is a form of API accessed over the web.

Type	Protocol	Data Format	Use Case
REST	HTTP	JSON / XML	Modern web/mobile APIs
SOAP	HTTP/SMTP	XML only	Enterprise, banking, legacy
GraphQL	HTTP	JSON	Flexible data querying
gRPC	HTTP/2	Protobuf	Microservices, low-latency

Advantages of Web Services

- Platform-independent — any language or OS can consume them
- Reusability — single service used by multiple clients
- Interoperability — different applications can communicate
- Loose coupling — changes in one service don't break others
- Scalability — services can be scaled independently

8.6 Pre-requisites for API Testing

- **API Documentation** — Swagger/OpenAPI spec or Postman collection
- **Base URL / Environment** — Know which environment (dev/staging/prod) to test
- **Authentication credentials** — API keys, tokens, username/password
- **Test data** — Sample request bodies, expected responses
- **HTTP client tool** — Postman, Playwright, curl, RestAssured
- **Understanding of HTTP** — Methods, status codes, headers, body
- **Knowledge of JSON/XML** — Response parsing and validation
- **Test scenarios** — Happy path, negative tests, edge cases

Module 9: Postman

9.1 Overview of Postman

Postman is a popular API development and testing platform with a graphical interface. It allows developers and testers to design, build, test, and document APIs without writing code.

- **Collections:** Organize requests into groups/folders
- **Environments:** Manage variables across different environments (dev, staging, prod)
- **Tests (pm.test):** Write JavaScript-based assertions in the Tests tab
- **Pre-request Scripts:** Execute code before a request (generate tokens, set vars)
- **Monitors:** Schedule collections to run automatically
- **Newman:** CLI runner for Postman collections in CI/CD pipelines
- **Mock Servers:** Simulate API responses before backend is ready
- **API Documentation:** Auto-generate documentation from collections

9.2 Project 1 — Basic CRUD Operations

Using <https://reqres.in> as the test API:

Step 1: Create Collection & Set Base URL

- Open Postman → New Collection → Name: 'ReqRes CRUD Tests'
- Go to Variables tab → Add: baseUrl = <https://reqres.in/api>
- Use `{{baseUrl}}` in request URLs

Step 2: GET All Users

```
URL:    GET  {{baseUrl}}/users?page=1
```

Tests tab:

```
pm.test("Status is 200", function() {  
    pm.response.to.have.status(200);  
});  
  
pm.test("Response has data array", function() {  
    const body = pm.response.json();  
    pm.expect(body).to.have.property('data');  
    pm.expect(body.data).to.be.an('array');  
});  
  
pm.test("Response time < 2s", function() {  
    pm.expect(pm.response.responseTime).to.be.below(2000);  
});
```

Step 3: POST Create User

```
URL:    POST  {{baseUrl}}/users
```

Body (raw/JSON):

```
{  
  "name": "Test User",  
  "job": "QA Engineer"  
}
```

Tests tab:

```
pm.test("Status is 201 Created", function() {  
  pm.response.to.have.status(201);  
});  
pm.test("User ID is assigned", function() {  
  const body = pm.response.json();  
  pm.expect(body).to.have.property('id');  
  pm.environment.set("userId", body.id); // Save ID for next request  
});
```

Step 4: PUT Update User

URL: PUT {{baseUrl}}/users/{{userId}}

Body (raw/JSON):

```
{  
  "name": "Updated User",  
  "job": "Senior QA"  
}
```

Tests tab:

```
pm.test("Status is 200", function() {  
  pm.response.to.have.status(200);  
});  
pm.test("Name is updated", function() {  
  pm.expect(pm.response.json().name).to.eql("Updated User");  
});
```

Step 5: DELETE User

URL: DELETE {{baseUrl}}/users/{{userId}}

Tests tab:

```
pm.test("Status is 204 No Content", function() {  
  pm.response.to.have.status(204);  
});
```

9.3 Project 2 — Comparison of Expected vs Actual Results

This project demonstrates structured test design by explicitly defining expected values and comparing them to actual API responses.

Test Design Pattern

```
// Pre-request Script: Define expected values
pm.variables.set("expectedName", "Janet");
pm.variables.set("expectedEmail", "janet.weaver@reqres.in");
pm.variables.set("expectedId", 2);
// Tests tab: Compare expected vs actual
const response = pm.response.json();
const expected = {
  id: pm.variables.get("expectedId"),
  name: pm.variables.get("expectedName"),
  email: pm.variables.get("expectedEmail")
};

pm.test("ID matches expected", function() {
  pm.expect(response.data.id).to.eql(Number(expected.id));
});
pm.test("First name matches expected", function() {
  pm.expect(response.data.first_name).to.eql(expected.name);
});
pm.test("Email matches expected", function() {
  pm.expect(response.data.email).to.eql(expected.email);
});
pm.test("Email format is valid", function() {
  pm.expect(response.data.email).to.match(/^[^\s@]+@[^\s@]+\.[^\s@]+$/);
});
pm.test("Content-Type is JSON", function() {
  pm.response.to.have.header("Content-Type");
  pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});
```

Using a Data-Driven Approach with CSV/JSON files

Postman's Collection Runner allows you to upload a CSV or JSON file and run the same request for multiple datasets. This enables comparison of expected vs. actual for many records simultaneously.

```
// expected_users.csv
userId,expectedFirstName,expectedEmail
1,George,george.bluth@reqres.in
2,Janet,janet.weaver@reqres.in
3,Emma,emma.wong@reqres.in
```

```
// In test script, use:
```

```
pm.test("First name matches CSV data", function() {
```

```
    const body = pm.response.json();
```

```
    pm.expect(body.data.first_name).to.eql(pm.iterationData.get("expectedFirstName"));
```

```
});
```

Quick Reference Summary

Module	Key Concepts	Key Methods / Tools
1. Playwright Overview	Cross-browser, API + UI testing	request fixture, APIRequestContext
2. Basic API Testing	CRUD, headers, params, body	get/post/put/patch/delete, response.json()
3. Assertions	Status, body, headers, response time	expect().toBe/toEqual/toMatchObject
4. Authentication	Basic, Bearer, OAuth2, API Key	Authorization header, httpCredentials
5. Test Data	Static/dynamic, chaining, env vars, JSON files	process.env, JSON.parse, faker
6. CI/CD	GitHub Actions, Jenkins pipeline	npx playwright test, environment secrets
7. Web Fundamentals	App types, 3-tier architecture	Frontend, Backend, Database
8. HTTP, JSON, APIs	Status codes, methods, SOA, Web Services	GET/POST/PUT/DELETE, REST, SOAP, GraphQL
9. Postman	Collections, CRUD, expected vs actual	pm.test, pm.expect, Newman, Collection Runner